Lec 8: Fault Tolerance

Naveen Kumar

Assistant Professor
SVNIT, Surat

April 18, 2026

Table of Contents

1. Introduction to fault tolerance
2. Process resilience
3. Reliable client-server communication
4. Reliable group communication
5. Distributed commit
6. Recovery
7. Summary

Partial Failures, Resilience, & Consensus

- **Partial Failure**: One component halts while others continue (unique to distributed systems).
- **Fault Tolerance Goal**: Automatically recover from partial failures transparently to the client.

Process Resilience via Groups

Identical processes are organized into **process groups**. To clients, they act as a single, highly-available process; if one crashes, others seamlessly take over.

The Consensus Challenge

Process groups must reach **consensus** on operations. We will explore **Paxos**, a widely-used algorithm for reaching agreement despite failures.

Reliable Communication, Commit, & Recovery

- **Reliable Group Communication**: Fault tolerance requires robust protocols beyond basic client-server models.
- **Atomic Multicasting**: Messages are delivered to **all** non-faulty processes or **none**, simplifying fault-tolerant design.

Distributed Commit Protocols

Coordinates processes to either **jointly commit** their work (make permanent) or **collectively abort** (revert to a safe state).

System Recovery

System states are periodically saved (e.g., via checkpoints) to guarantee accurate recovery to a correct state after a failure.

Table of Contents

1. Introduction to fault tolerance
2. Process resilience
3. Reliable client-server communication
4. Reliable group communication
5. Distributed commit
6. Recovery
7. Summary

The Foundation of Fault Tolerance: Dependability

Fault tolerance aims to achieve **dependability**, providing assurances across four core requirements:

- **Availability:** The probability the system is ready to use at *any given instant*.
- **Reliability:** The ability to run continuously without failure over a *specific time interval*.
- **Safety:** Ensuring no catastrophic events occur when the system temporarily fails.
- **Maintainability:** How easily and quickly a failed system can be repaired.

Availability vs. Reliability

The crucial difference lies in **The Time Factor**:

- **Availability** is measured at a specific *instant*.
- **Reliability** is measured over a continuous *interval*.

Illustrative Example

- **High Availability, Low Reliability:** Goes down for 1ms every hour. Highly available ($> 99.9999\%$), but unreliable for continuous execution.
- **High Reliability, Lower Availability:** Never crashes, but offline for two weeks every August. Perfectly reliable when running, but lower overall availability ($\sim 96\%$).

Traditional Metrics and Reliability Math

Traditionally, fault tolerance is quantified using three time-based metrics:

- **MTTF (Mean Time To Failure):** Average time until a component fails.
- **MTTR (Mean Time To Repair):** Average time needed to repair it.
- **MTBF (Mean Time Between Failures):** Simply MTTF + MTTR.

Long-term availability (A) is calculated as:

$$A = \frac{\text{MTTF}}{\text{MTBF}} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}$$

Assuming a component does not age (constant failure rate z), its reliability over time t follows an exponential distribution $R(t) = e^{-zt}$.

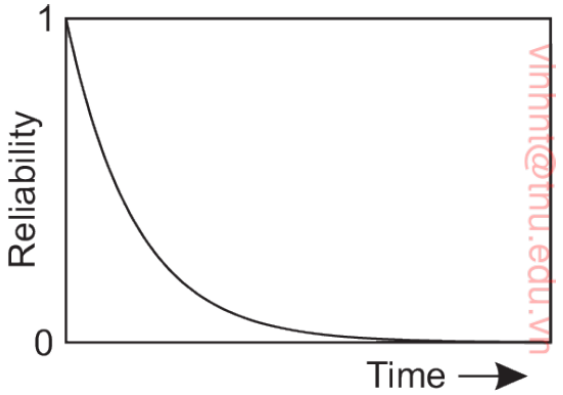


Figure 1: The reliability of a component having a constant failure rate.

The Chain of Malfunction: Fault, Error, and Failure

System breakdowns operate as a chain reaction:

- **Fault:** The root cause (e.g., programming bug, degraded cable).
- **Error:** A compromised state that may lead to a failure (e.g., a flipped bit).
- **Failure:** The observable outcome where the system fails to provide its services (e.g., a crashed program).

The Causal Chain: Fault $\rightarrow$ Error $\rightarrow$ Failure
Fault: free(ptr) omitted $\rightarrow$ Error: Heap exhaustion $\rightarrow$ Failure: OOM crash
Core Objective: Intercept this chain to maintain services despite active faults and errors.

Fault Classifications & Recovery

Faults are the underlying cause of **errors**, which may lead to system **failure**. They are categorized by their temporal behavior:

- **Transient Faults:** Occur once and disappear.
  - *Example:* A bird flying through a microwave beam causing lost bits.
  - *Handling:* **Time Redundancy** (Retrying the operation).
- **Intermittent Faults:** Appear, vanish, and reappear unpredictably.
  - *Example:* A loose contact on a hardware connector.
  - *Handling:* Difficult to diagnose; often requires monitoring/redundancy.
- **Permanent Faults:** Persist until the component is replaced or patched.
  - *Example:* Burnt-out chips, software bugs, or disk-head crashes.
  - *Handling:* **Physical Redundancy** (Extra equipment or processes).

The Goal of Fault Tolerance

To use **Redundancy** (Information, Time, or Physical) to mask the presence of faults from the user.

Failure Detection & System Synchrony

The ability to distinguish a crashed process from a slow one depends on system synchrony:

- **Synchronous System:** Process speeds and message delays are strictly bounded.
  - *Detection:* **Fail-stop** behavior. P can rightfully conclude Q has crashed if a timeout occurs.
- **Asynchronous System:** No assumptions about execution speeds or delivery times.
  - *Detection:* **Fail-silent** behavior. P cannot distinguish a crash from a message that is simply slow or lost.

Practical Reality (Partial Synchrony): Most systems behave synchronously most of the time, but have no bound on asynchronous periods.

The Detection Challenge

In practice, timeouts are crude and produce **false positives**. Fault-tolerant protocols must be designed to withstand incorrectly detecting a process as halted.

Different types of failures

Type of failure	Description of server's behavior
Crash failure	Halts, but is working correctly until it halts
Omission failure	Fails to respond to incoming requests
Receive omission	Fails to receive incoming messages
Send omission	Fails to send messages
Timing failure	Response lies outside a specified time interval
Response failure	Response is incorrect
Value failure	The value of the response is wrong
State-transition failure	Deviates from the correct flow of control
Arbitrary failure	May produce arbitrary responses at arbitrary times

Figure 2: Different types of failures

The Spectrum of Halting Failures

Halting failures can be classified from the least to the most severe based on our ability to detect them correctly:

- **Fail-stop:** Crash failures that can be instantly and reliably detected (assuming worst-case delay bounds).
- **Fail-noisy:** Crashes are detected, but the process may only *eventually* come to the correct conclusion (unreliable detection window).
- **Fail-silent:** The system cannot distinguish a true crash failure from a communication omission failure.
- **Fail-safe:** The server exhibits arbitrary failures, but they are benign and do not harm the overall system state.
- **Fail-arbitrary:** The server fails in completely unobservable, arbitrary, and highly harmful ways (Byzantine behavior).

Failure Masking by Redundancy

Fault tolerance relies on hiding (masking) failures using **redundancy**:

- **Information Redundancy:** Adding extra bits to recover garbled data (e.g., Hamming codes).
- **Time Redundancy:** Retrying failed actions. Highly effective against transient faults (e.g., network retransmissions).
- **Physical Redundancy:** Using hardware or software replicas to take over if components crash.

Example: Triple Modular Redundancy (TMR)

A classic example of physical redundancy in electronic devices is Triple Modular Redundancy.

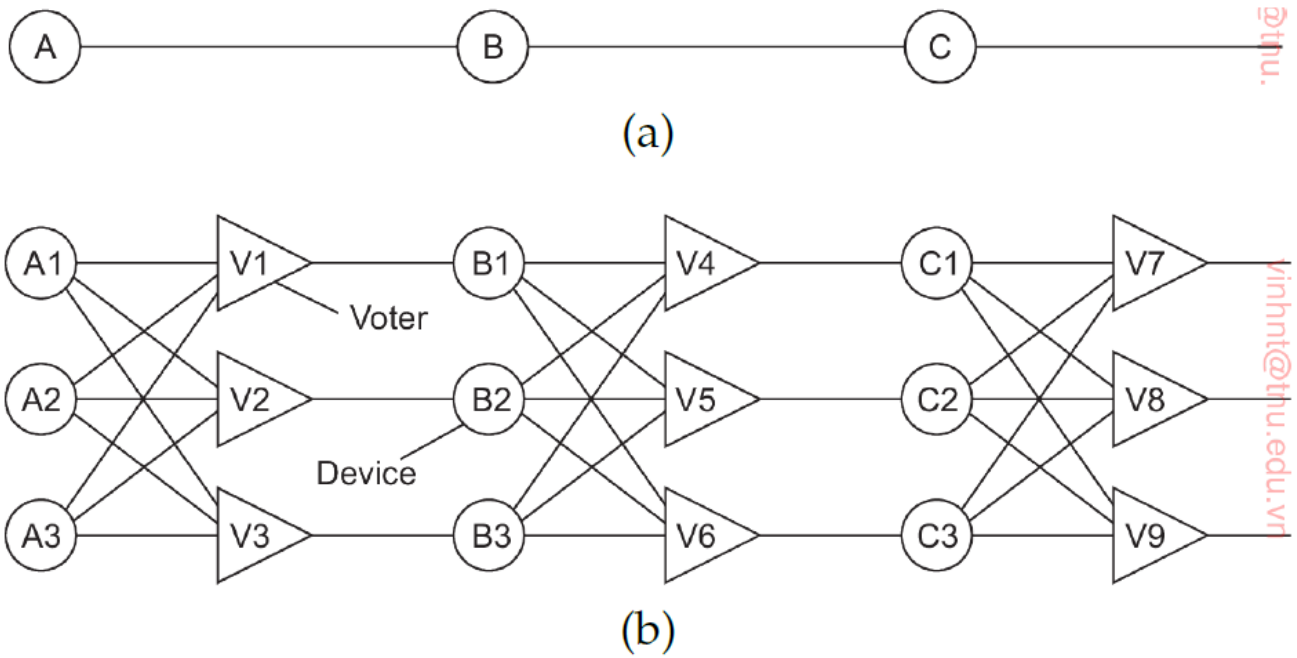


Figure 3: Triple modular redundancy

How it Works: Every component is triplicated. Their outputs pass through a **voter** circuit. If one component fails (e.g., outputs a rogue value), the voter takes the majority view of the two correct inputs. The fault is completely masked from the next stage of the system.

Table of Contents

1. Introduction to fault tolerance
2. Process resilience
3. Reliable client-server communication
4. Reliable group communication
5. Distributed commit
6. Recovery
7. Summary

Process Resilience via Groups

The primary approach to tolerating a faulty process is organizing identical processes into a **process group**.

The Group Abstraction

- When a client sends a message to the group, all members receive it.
- To the outside world, the group appears as a *single, logical process*.
- If one process fails, another silently takes over its workload without the client noticing.

Dynamic Nature

- Groups are not static. Processes can join or leave during system operation, requiring robust membership management.

Group Organization

Process groups generally follow one of two internal structures, each with distinct trade-offs.

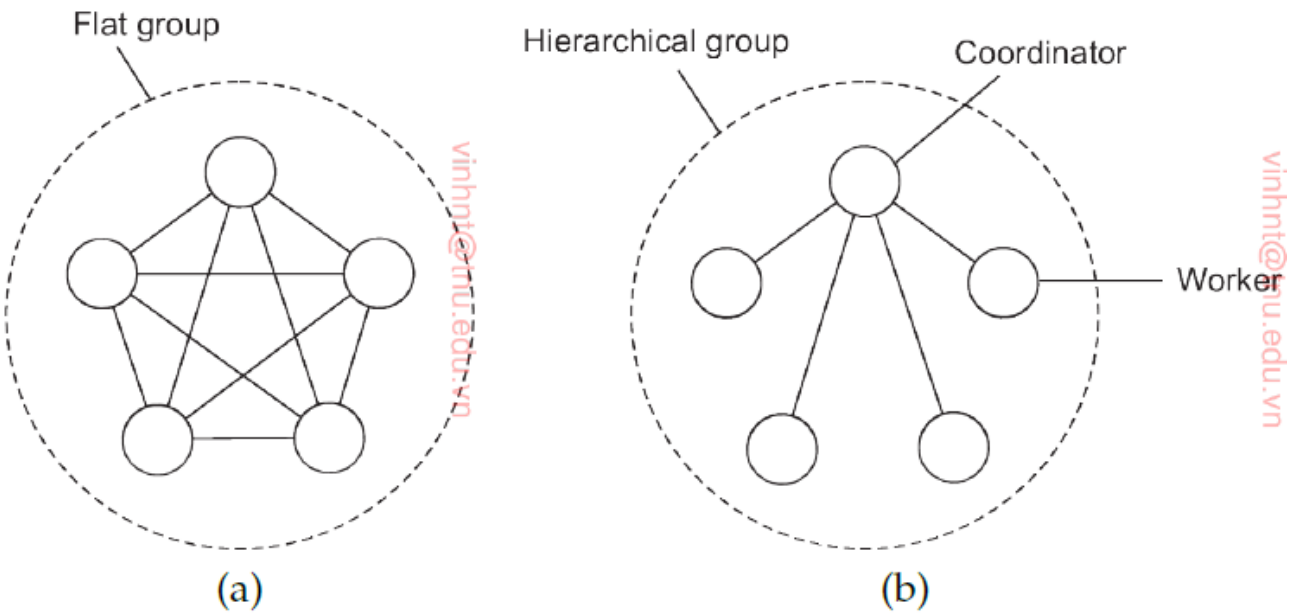


Figure 4: Communication in a (a) flat group and in a (b) hierarchical group.

Flat Group: All processes are equal.

- *Advantage:* No single point of failure.
- *Disadvantage:* Decision making is complex and incurs delay (requires voting).

Membership Management

How do we track who is currently in the group?

Centralized Approach

- A single "group server" maintains the database of all groups and members.
- Efficient, but introduces a single point of failure.

Distributed Approach

- Uses reliable multicasting to announce joins.
- *The Leave Problem:* Voluntary leaves are polite, but crashes are silent. The group must rely on timeouts to detect a crashed member experimentally.
- *Synchrony Requirement:* Joins and leaves must be perfectly synchronized with the data message stream to ensure state consistency.

Failure Masking and Replication Constraints

To mask failures, we use primary-backup (hierarchical) or replicated-write (flat, active replication) protocols.
But how much replication is needed?

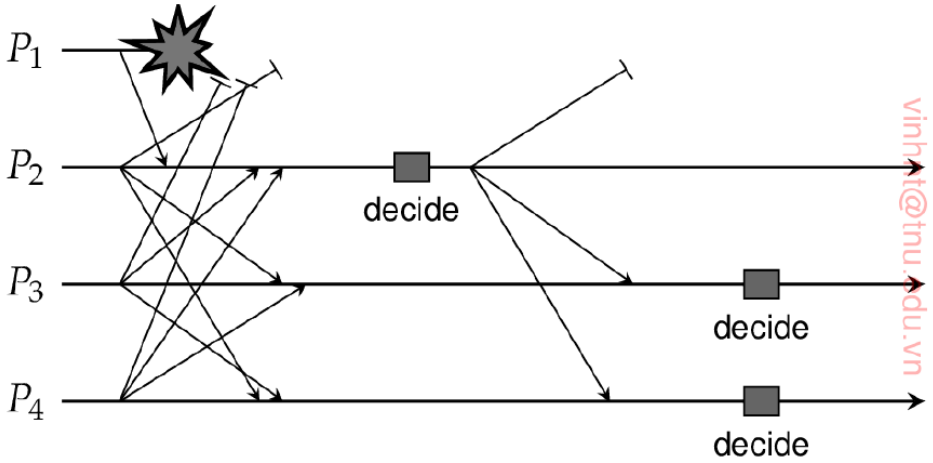
A system is **k-fault tolerant** if it can survive faults in k components and still meet specifications.

Replication Requirements:

- **Crash (Silent) Failures:** Requires a minimum of $\mathbf{k + 1}$ processes. If k processes stop, the 1 remaining process provides the correct answer.
- **Arbitrary (Byzantine) Failures:** Requires a minimum of $\mathbf{2k + 1}$ processes. If k faulty processes intentionally output the same wrong answer, the remaining $k + 1$ nonfaulty processes will form a correct majority vote.

Consensus with Crash Failures

To act as a single logical process, group members must reach **consensus** on which command to execute, and in what order.



vinhkt@tnu.edu.vn

Figure 5: Reaching consensus through flooding in the presence of crash failures (Cachin et al.)

Flooding Consensus Algorithm & The $k+1$ Rule

- **The Mechanism:** Operates in synchronous rounds. Processes broadcast all known proposed commands to the group.
- **The Partial Broadcast Problem:** As shown, if P_1 crashes mid-round, it might successfully send data to only P_2, failing before it can reach P_3 or P_4.
- **The Worst-Case Scenario:** If P_2 tries to forward this unique data in the next round but also crashes, the information could be lost. Nodes cannot safely commit early.
- **The Theoretical Guarantee:** To tolerate up to k crash failures, the system must execute exactly $k+1$ **rounds**. This guarantees at least one clean round where all survivors synchronize identical data.

Introduction to Paxos

Paxos is a highly robust consensus algorithm designed to guarantee safety in chaotic, imperfect networks where messages drop and servers crash.

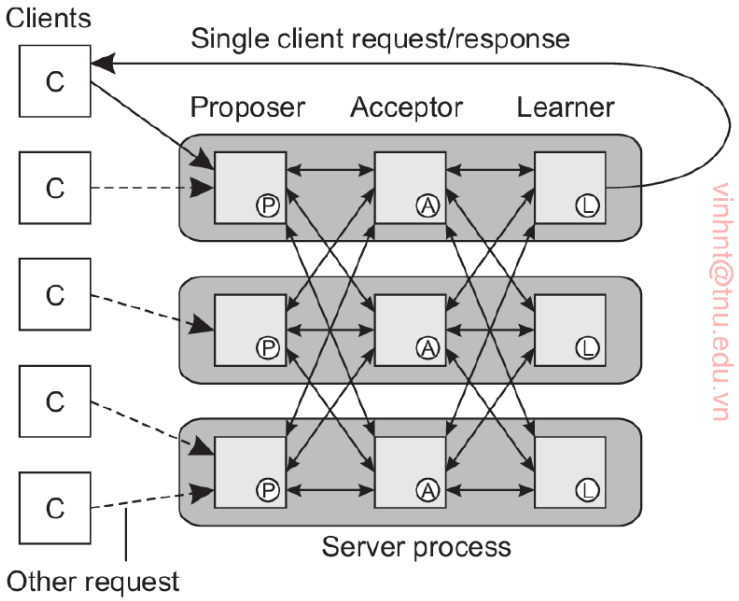


Figure 6: The organization of Paxos into different logical processes.

The Three Logical Roles (often running on the same physical server):

- *Proposer (The Advocate):* Receives client requests and drives the consensus process to get them accepted.
- *Acceptor (The Voter):* Forms the democratic core. They vote on proposals. A **majority** must agree for any progress to occur.
- *Learner (The Executor):* Passively waits for a majority agreement, then applies the operation to the local state.

The Paxos Algorithm: Phase 1 (Prepare)

To prevent multiple "leaders" from causing chaos, Paxos uses globally ordered timestamps (proposal numbers).

Phase 1a: The Prepare Request

- The Proposer selects a unique, strictly higher timestamp t and broadcasts `prepare(t)` to all Acceptors.
- *The Goal:* Secure the floor. It asks: "Will you promise to ignore older leaders and tell me about any past decisions?"

Phase 1b: The Promise Response

- If t is the highest timestamp an Acceptor has ever seen, it replies with `promise(t)`.
- If the Acceptor previously accepted an older operation, it MUST include that operation in its promise.
- If the Acceptor has already seen a higher timestamp than t, it silently ignores the request.

The Paxos Algorithm: Phase 2 (Accept)

Once a Proposer collects promises from a **majority** of Acceptors, it assumes leadership and proceeds.

Phase 2a: The Accept Request

- *Clean Slate:* If Acceptors reported no prior operations, the Proposer sends `accept(t, o)` with its own new operation o.
- *The Adoption Rule:* If Acceptors reported a previously accepted operation o', the Proposer MUST abandon its own idea and *adopt* o', sending `accept(t, o')`. This prevents new leaders from overwriting established consensus.

Phase 2b: Learn and Execute

- Acceptors log the `accept` message and broadcast `learn(o)` to all Learners (unless they were interrupted by a newer Phase 1).
- Once a Learner counts a **majority** of `learn(o)` messages, the operation is officially executed.

Evolution of Paxos: The 2-Server Case

To understand Paxos, we first examine a simple 2-server primary-backup setup under normal, fault-free conditions.

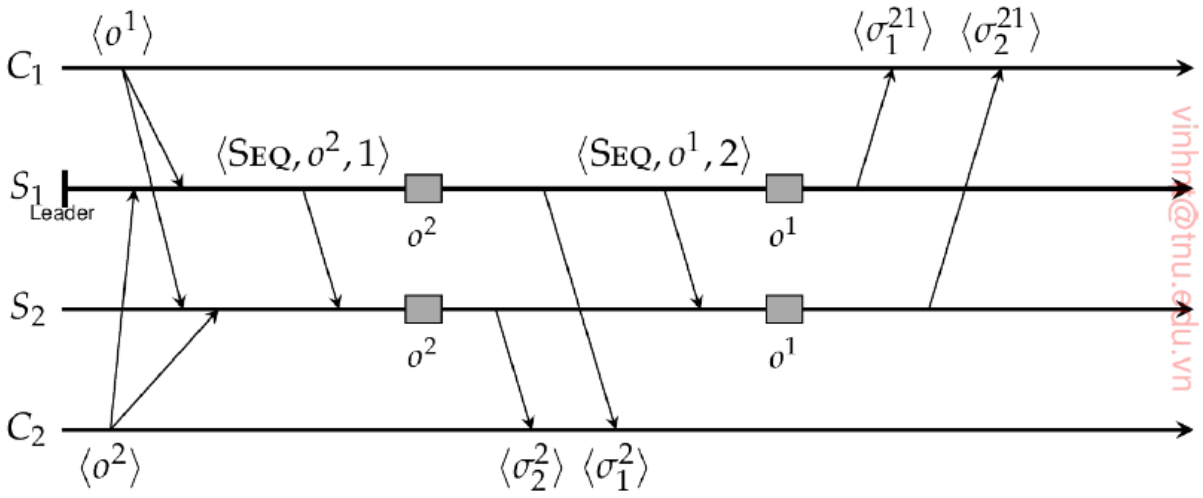
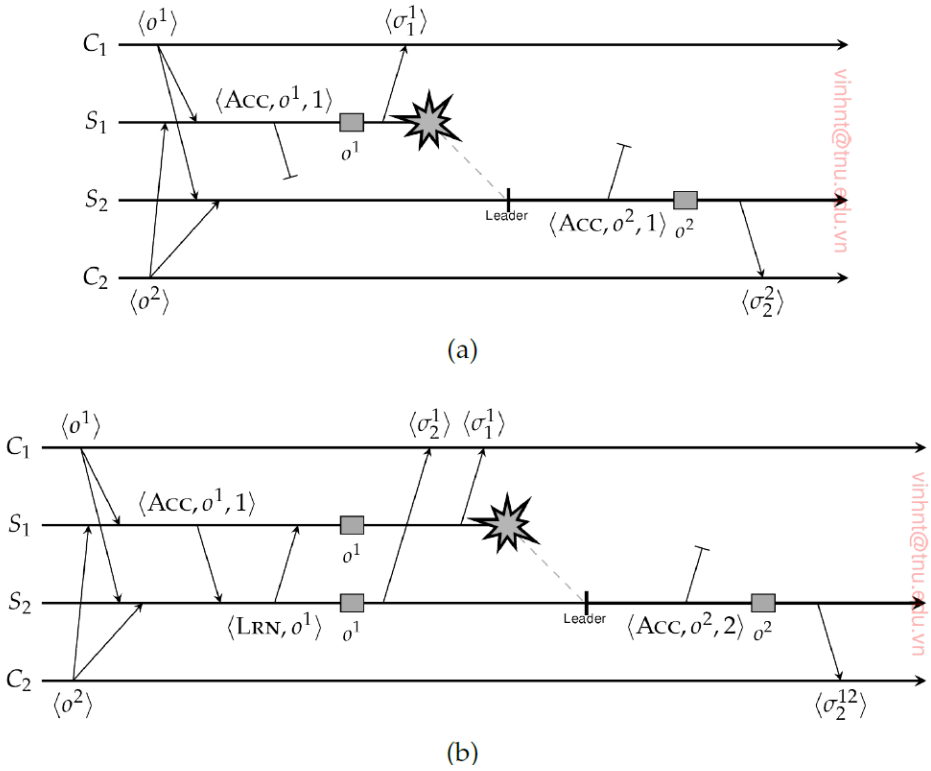


Figure 7: Two clients communicating with a 2-server process group.

- One server (S_1) acts as the primary sequencer, assigning a unique timestamp to every submitted command.
- Servers are required to execute commands according to their assigned timestamp.
- S_1 instructs S_2 to execute operations by sending an `ACCEPT` message, ensuring both maintain identical states.
- This simple replication mechanism works flawlessly provided the network and hardware remain perfectly reliable.

Evolution of Paxos: Handling Leader Crashes

- The worst-case scenario in a 2-server setup:
  - The leader executes an operation locally.
  - The ACCEPT message to the backup is **lost**.
  - The **leader crashes** immediately after.
- The surviving backup takes over, but it is missing data. The system state is now **inconsistent**.
- **The Paxos Safety Fix:**
  - A server may execute an operation *only if* it knows that the **other server has learned the operation as well**.



(a) Failure scenario: lost accept + leader crash
(b) Safe solution with learning guarantee [cite: 1]

Evolution of Paxos: Expanding to Three Servers

Adding a third server requires scaling the "learning" rule to prevent divergence during multiple failures.

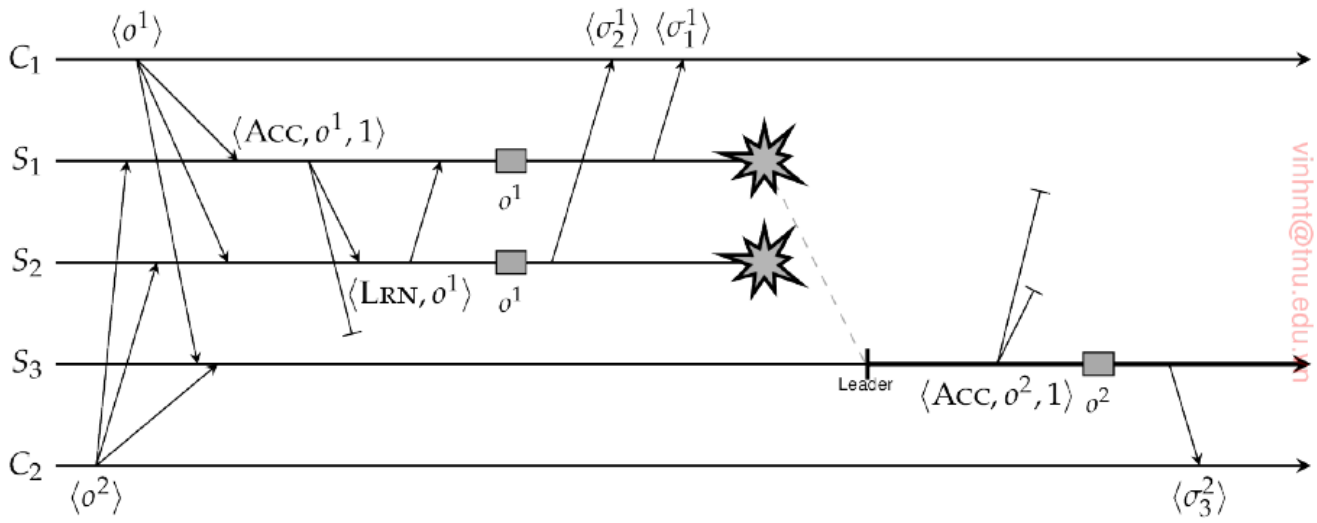


Figure 8: The situation when dealing with three servers of which two crash.

- Imagine S_1 (leader) updates S_2, but the `ACCEPT` message to S_3 is lost. If S_1 and S_2 subsequently crash, S_3 takes over leadership without knowing about the previous operation.
- **The Refined Rule:** In Paxos, a server S cannot execute an operation until it has received a `LEARN` message from **all** other nonfaulty servers.

Evolution of Paxos: False Failure Detections

In partially synchronous systems, timeouts are used for failure detection, which can lead to false positives where a slow process is incorrectly assumed dead.

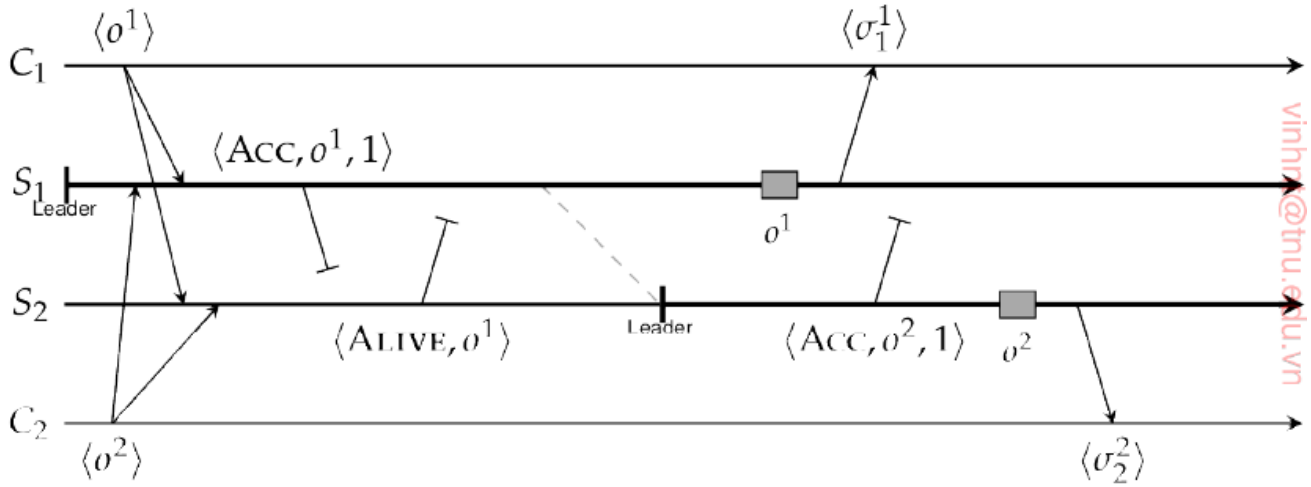


Figure 9: The situation in the case of false failure detections.

- If S_2 falsely detects S_1 as crashed, both might independently decide to execute their operation of choice, leading to divergent behavior.
- **The Majority Mandate:** To prevent this split-brain scenario, a server can execute an operation only if it is certain that a **majority** will execute that same operation.

Evolution of Paxos: Incorporating Leader IDs

Even with majorities, heavily delayed messages from deposed "zombie" leaders can arrive out of nowhere and corrupt the current state.

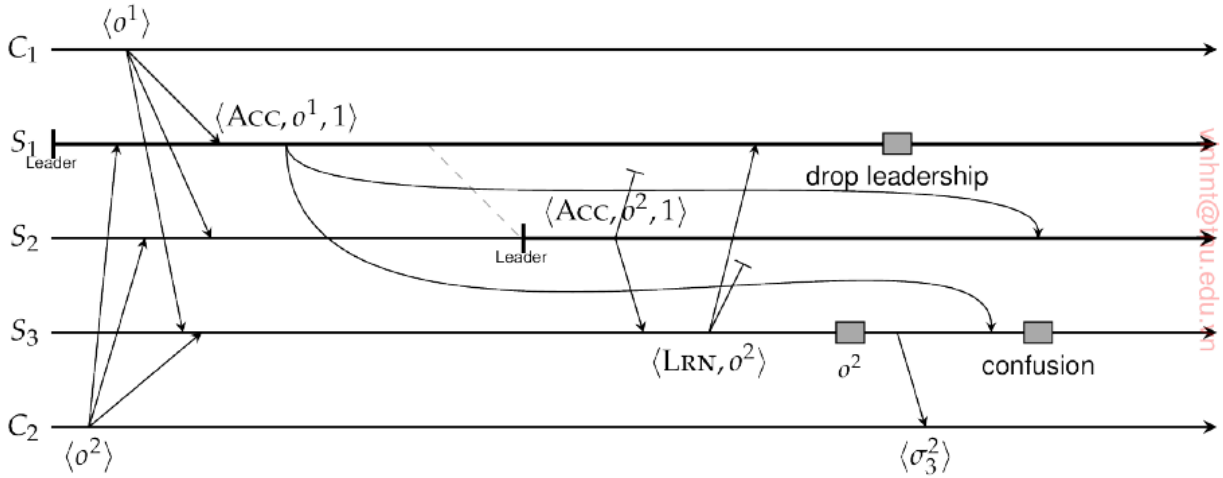


Figure 10: Why incorporating the ID of the current leader is needed.

- Suppose S_2 takes over from a lagging S_1. If an old `ACCEPT` message from S_1 finally reaches S_3, S_3 must know it is stale.
- **The Solution:** The leader must include its ID in an `ACCEPT` message. Combined with deterministic leader election, backups can safely reject messages from former leaders.

Understanding Paxos: Progress and Liveness

Paxos formally ensures **Safety** (only proposed operations are learned, and at most one at a time), but **Liveness** (eventual progress) requires additional steps.

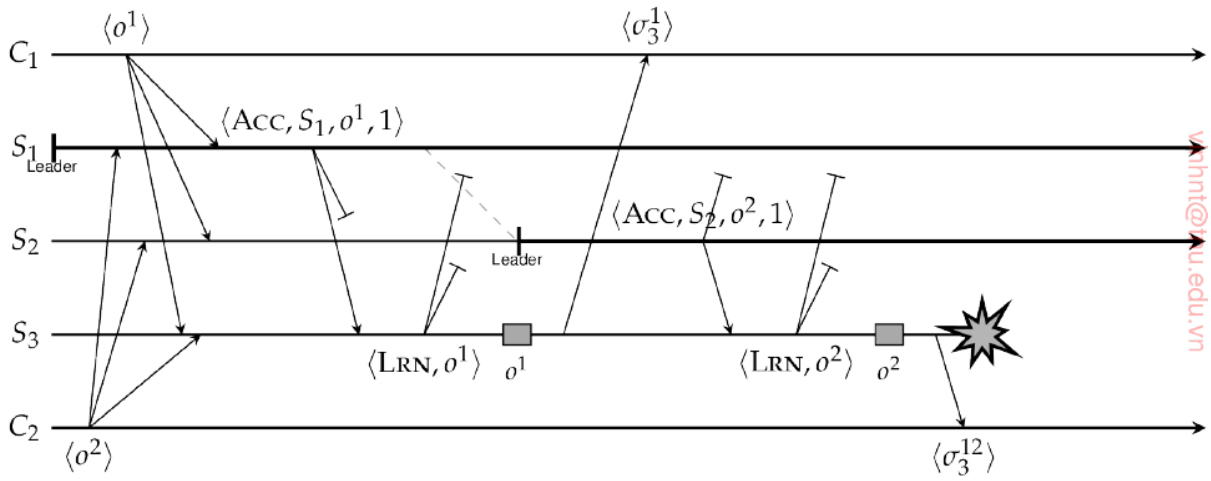


Figure 11: When Paxos can make no further progress due to lack of consensus on the leader.

- **The Livelock Problem:** If servers have no consensus on who the leader is, progress halts. For example, lost `LEARN` messages can leave servers permanently confused about what was actually executed.
- **Practical Fix:** Implement an explicit leadership takeover (e.g., broadcasting a `PROPOSE` message). A new leader must ensure any outstanding operations are properly dealt with before initiating new ones.

Deconstructing Paxos: Why Every Piece Matters

Paxos seems complex, but every mechanism exists to prevent a specific failure scenario in a partially synchronous network.

The Leader (Sequencer): *Why?* To enforce a strict, single chronological order for all client requests.
→ Prevents: State inconsistency caused by servers executing the same commands in different orders.

Majority Quorums: *Why?* An operation is only "chosen" if accepted by a majority. Since majorities always overlap, two different operations cannot be chosen.
→ Prevents: "Split-brain" scenarios where a network partition causes two sub-groups to act independently.

Logical Timestamps: *Why?* To uniquely identify proposals and track chronological progress.
→ Prevents: Proposers who falsely believe they are the leader from successfully pushing stale or conflicting proposals.

The "Learn" Requirement: *Why?* A server will not actually *execute* an operation until it knows a majority has also learned it.
→ Prevents: The fatal flaw where a leader updates one backup and then crashes, leaving the system with an isolated, unrecoverable state.

Leader IDs in Messages: *Why?* To explicitly tie an `ACCEPT` message to a specific leader's reign.
→ Prevents: Heavily delayed messages from deposed "zombie" leaders from arriving late and corrupting the new leader's operations.

Consensus with Arbitrary (Byzantine) Failures

Standard Paxos handles *crash* failures. But what if a server is hacked, corrupted, or actively lies? This is an arbitrary or **Byzantine** failure.

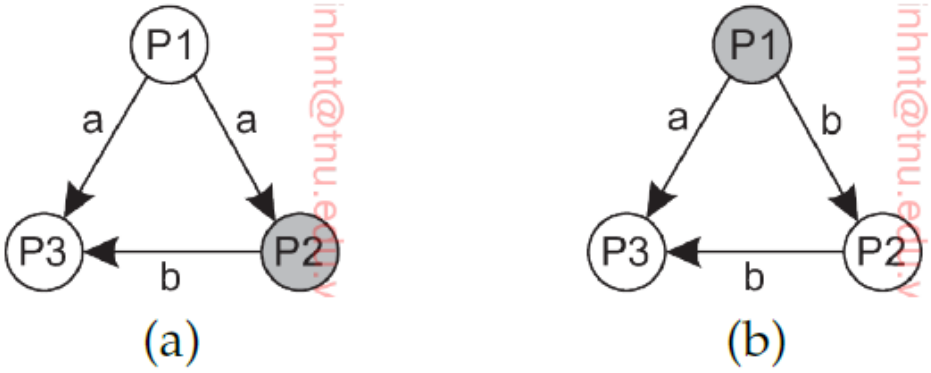


Figure 12: A process in a replicated group acting inconsistently: (a) not forwarding properly, and (b) telling different things to different processes.

Strict Rules for Byzantine Agreement:

- **Consistency (BA1):** Every honest, nonfaulty node must eventually store the exact same value.
- **Fidelity (BA2):** If the Primary node is honest, every honest backup must store exactly what the Primary initially proposed.

Why 3k Processes are NOT Enough

To survive k Byzantine (lying) failures, having $3k$ processes is mathematically insufficient. Consider $k = 1$ (one liar) in a 3-node system:

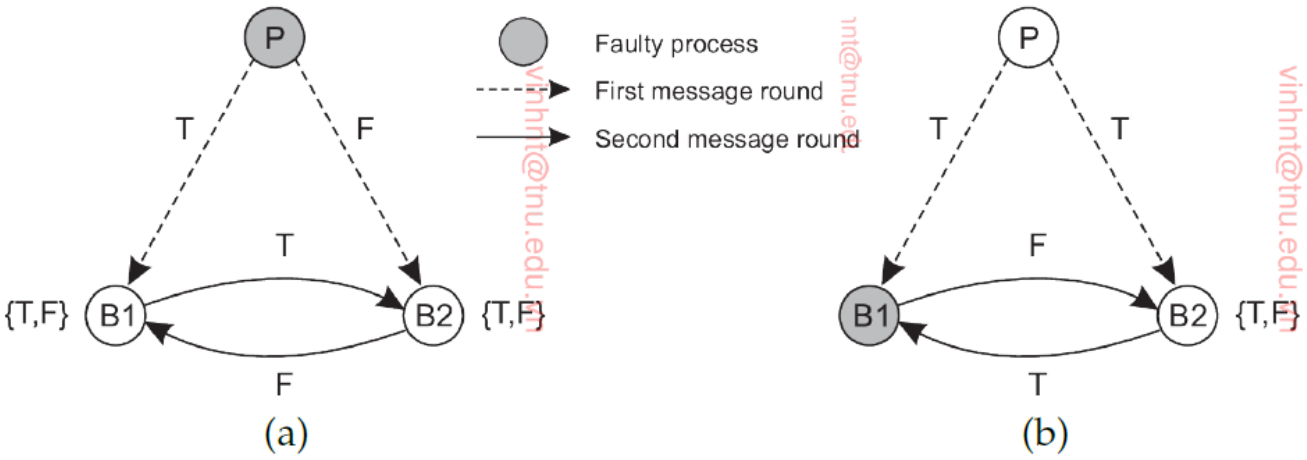


Figure 13: Impossibility to reach consensus with 3 processes and trying to tolerate a single arbitrarily failing process.

The "He Said, She Said" Deadlock:

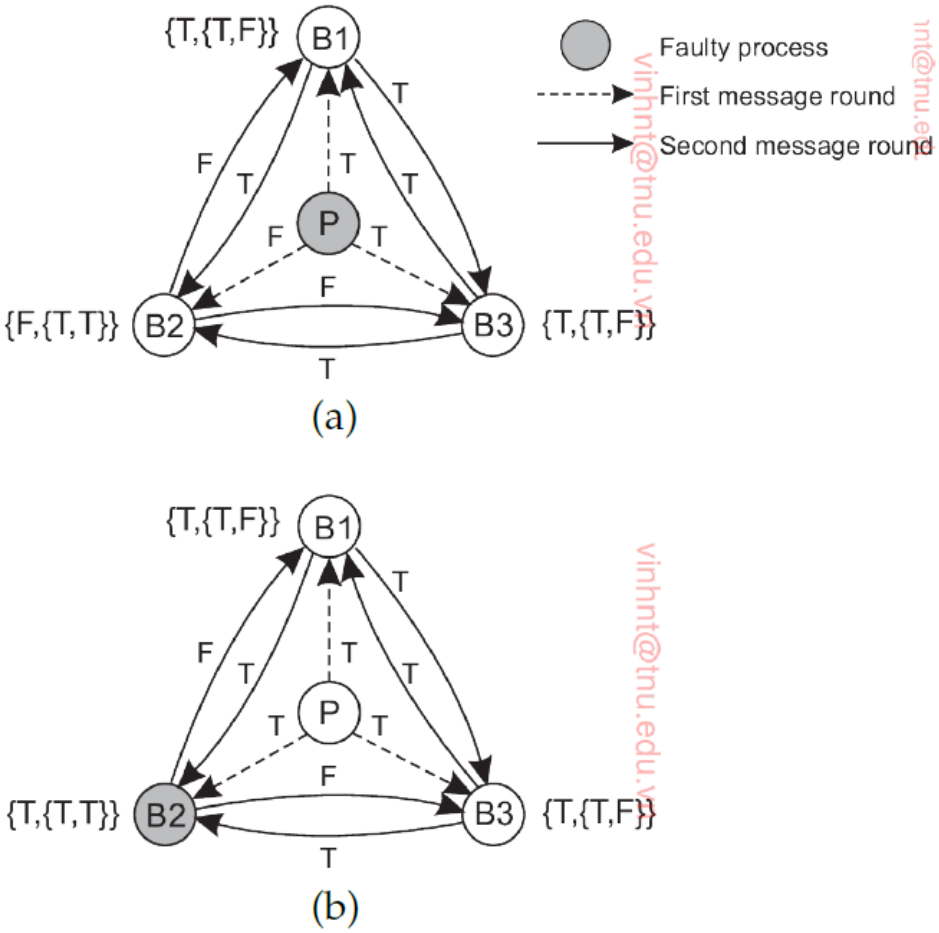
- The Primary maliciously sends "True" to B_1 and "False" to B_2.
- B_1 and B_2 forward their conflicting data to each other.
- Both honest backups now have conflicting data $\{T, F\}$. Without a tie-breaker, they cannot determine if the Primary lied, or if the other backup is lying.

Why 3k + 1 Processes ARE Enough

- To safely outvote k malicious nodes, the system size must be:

$$n \geq 3k + 1$$

- *Example ($k = 1$):* We need **4 nodes** to tolerate 1 traitor.
- **How It Breaks the Tie:**
  - Even if the Primary lies to different nodes...
  - The three backups cross-check their received data.
- After sharing, honest nodes construct a view like: $\{T, T, F\}$



Consensus with 4 processes where 1 may behave arbitrarily

vinhnt@tnu.edu.vn

Limitations: The Fischer-Lynch-Paterson (FLP) Impossibility

Distributed consensus is incredibly hard. Under realistic network conditions, it is actually mathematically impossible to guarantee.

Process behavior	Communication delay	Message ordering: Unordered, Unicast	Unordered, Multicast	Ordered, Unicast	Ordered, Multicast
Synchronous	Bounded	X	X	X	X
	Unbounded			X	X
Asynchronous	Bounded				X
	Unbounded				X

Message transmission

Figure 14: The matrix of consensus possibilities. Notice the failure of Unicast in asynchronous systems.

The Fischer-Lynch-Paterson (FLP) Theorem

- Core Result: Consensus is impossible in an asynchronous system if even one process crashes silently.
- The Root Cause: Surviving nodes cannot distinguish between a crashed server and extreme network delay.
- The Dilemma: Waiting indefinitely halts the system (losing Liveness), while proceeding prematurely risks fracturing the state (losing Safety).
- Practical Impact: Algorithms like Paxos, Raft, and PBFT cannot strictly guarantee termination; they rely on timeouts as real-world workarounds.

The CAP Theorem

The CAP theorem formalizes the physical limitations of distributed systems subject to network failures (partitions). A networked shared-data system can only provide **two** of the following three:

- **Consistency (C):** All nodes see the same, up-to-date data at the same time.
- **Availability (A):** Every request receives a response (updates are always executed).
- **Partition Tolerance (P):** The system operates despite network communication failures.

The Engineering Reality: Because networks *will* fail (Partitions happen), engineers must choose between preserving Consistency (pausing availability) or Availability (accepting temporary data inconsistency).

Failure Detection Subsystems

Fault tolerance requires detecting who has failed. This is traditionally managed via timeout mechanisms.

Detection Mechanisms

- **Active Probing:** Sending "Are you alive?" ping messages.
- **Passive Gossiping:** Nodes regularly broadcast their availability to neighbors. Missing recent gossip implies a failure.

The False Positive Problem

- In partially synchronous systems, a timeout might just mean network congestion. Removing a healthy process degrades system health.
- *Solution:* Use "Eventually perfect failure detectors" that reinstate nodes if delayed messages finally arrive, and dynamically increase timeout thresholds.

Table of Contents

1. Introduction to fault tolerance
2. Process resilience
3. Reliable client-server communication
4. Reliable group communication
5. Distributed commit
6. Recovery
7. Summary

Reliable Communication Overview

While process resilience handles faulty processes, distributed systems must also address communication failures.

Failure Types in Communication

- Channels can exhibit crash, omission, timing, and arbitrary failures.
- Arbitrary failures often manifest as duplicate messages due to long network buffering.

Point-to-Point Communication (e.g., TCP)

- *Omission Failures:* TCP successfully masks lost messages using acknowledgments and retransmissions.
- *Crash Failures:* TCP cannot mask an abruptly broken connection. The client is usually informed via an exception, and the only solution is attempting to set up a new connection.

RPC Semantics in the Presence of Failures

The primary goal of a Remote Procedure Call (RPC) is to hide communication and make remote calls look exactly like local ones. This transparency breaks down when errors occur.

We can distinguish between five different classes of RPC failures:

- The client is unable to locate the server.
- The request message from the client to the server is lost.
- The server crashes after receiving a request.
- The reply message from the server to the client is lost.
- The client crashes after sending a request.

Failure 1: Client Cannot Locate the Server

This occurs when all servers are down or if there is a version mismatch between the client stub and the newly evolved server interface.

Possible Solution

- Raise an error or an exception (e.g., a special signal handler in C or an exception in Java).

The Drawback

- It destroys the transparency we are trying to achieve.
- In a single-machine system, a programmer does not need to write an exception handler just in case a local procedure is "not there today." Forcing this in RPC breaks the illusion of a local call.

Failure 2: Lost Request Messages

Dealing with a lost request from the client to the server is the most straightforward failure to handle.

The Solution

- The client's operating system or stub starts a timer when the request is sent.
- If the timer expires before a reply or an acknowledgment is received, the client simply sends the message again.

Outcome

- If the message was truly lost, the server will treat the retransmission as the original request, and everything works perfectly.
- (If the message was not lost, we encounter the "lost reply" problem, discussed shortly).

Failure 3: Server Crashes

A severe complication arises when the server crashes while handling an RPC.

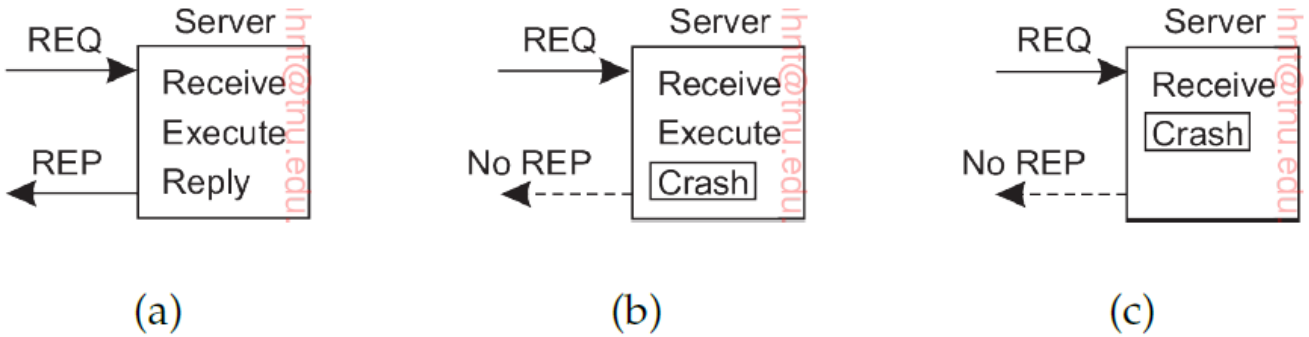


Figure 15: A server in client-server communication. (a) The normal case. (b) Crash after execution. (c) Crash before execution.

The Core Problem: When the client's timer expires, it cannot know which scenario occurred. Should it report a failure (scenario b) or retransmit the request (scenario c)?

Server Crashes: Three Philosophies

Because the client cannot determine when the server crashed, three philosophies exist for handling the situation:

- **At-least-once semantics:** Keep trying until a reply is received. Guarantees the RPC is carried out at least one time, but possibly multiple times.
- **At-most-once semantics:** Give up immediately and report a failure. Guarantees the RPC is carried out at most one time, but possibly not at all.
- **Guarantee nothing:** The client gets no help and no promises. The RPC may have executed zero times or many times. Easy to implement but unpredictable.

Note: "Exactly-once" semantics is what we actually want, but it is generally impossible to arrange in the face of crashes.

Failure 4: Lost Reply Messages

If a client's timer expires, it retransmits. But if the original request was executed and only the *reply* was lost, retransmitting causes the server to execute the request a second time.

Idempotent vs. Nonidempotent Operations

- *Idempotent:* Operations that can be safely repeated without harm (e.g., "Read the first 1024 bytes of a file").
- *Nonidempotent:* Operations that cause damage if repeated (e.g., "Transfer money from one account to another").

The Solution

- The client assigns a **sequence number** to each request.
- The server keeps track of the most recent sequence number from each client and refuses to carry out duplicate requests, instead just resending the cached reply.

Failure 5: Client Crashes (Orphans)

What happens if a client sends a request and then crashes before the server replies? The active computation on the server becomes an **orphan**.
Problems caused by Orphans:

- They waste valuable processing power on the server.
- They can lock files or tie up databases and system resources indefinitely.
- If the client reboots, reissues the RPC, and the reply from the old orphan arrives right after, it causes system confusion.

Dealing with Orphans

Four primary solutions have been proposed to manage orphan computations:

- **Orphan Extermination**: The client logs every RPC to disk before sending it. Upon reboot, it reads the log and sends a message to explicitly kill the orphan computation running on the server.
  - Drawback: High disk overhead, and it fails if the network is partitioned.
- **Reincarnation**: Time is divided into numbered "epochs". When a client reboots, it broadcasts a new epoch, causing servers to automatically kill any remote computations attached to older epochs.
- **Gentle Reincarnation**: Similar to standard reincarnation, but servers try to locate the computation's owner first. The computation is only killed if the owner cannot be found.
- **Expiration**: Each RPC is given a strict time quantum to finish. If a client crashes, it simply waits out that specific time limit before rebooting, ensuring all orphans have safely expired on their own.

Table of Contents

1. Introduction to fault tolerance
2. Process resilience
3. Reliable client-server communication
4. Reliable group communication
5. Distributed commit
6. Recovery
7. Summary

Reliable Group Communication

Reliable group communication guarantees that messages are delivered to all members in a process group.
To understand how this is achieved, we must separate the network logic from the application logic by distinguishing between **receiving** and **delivering** a message.

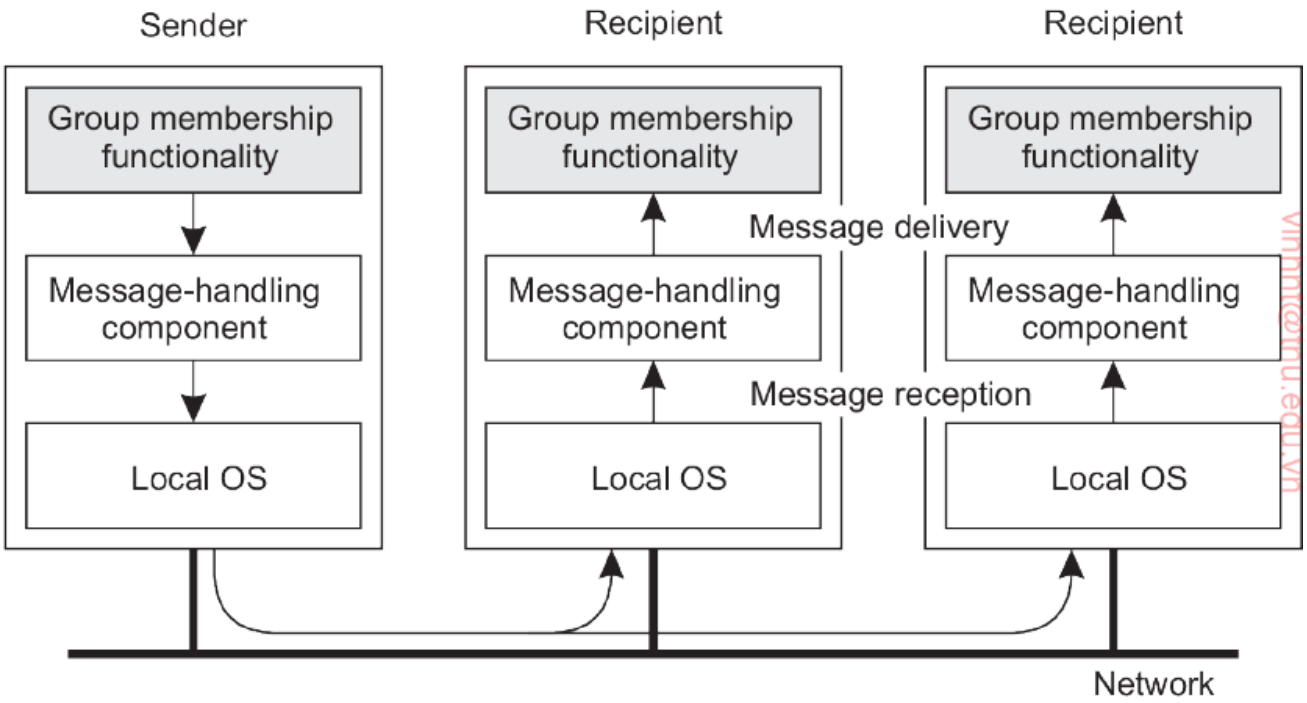


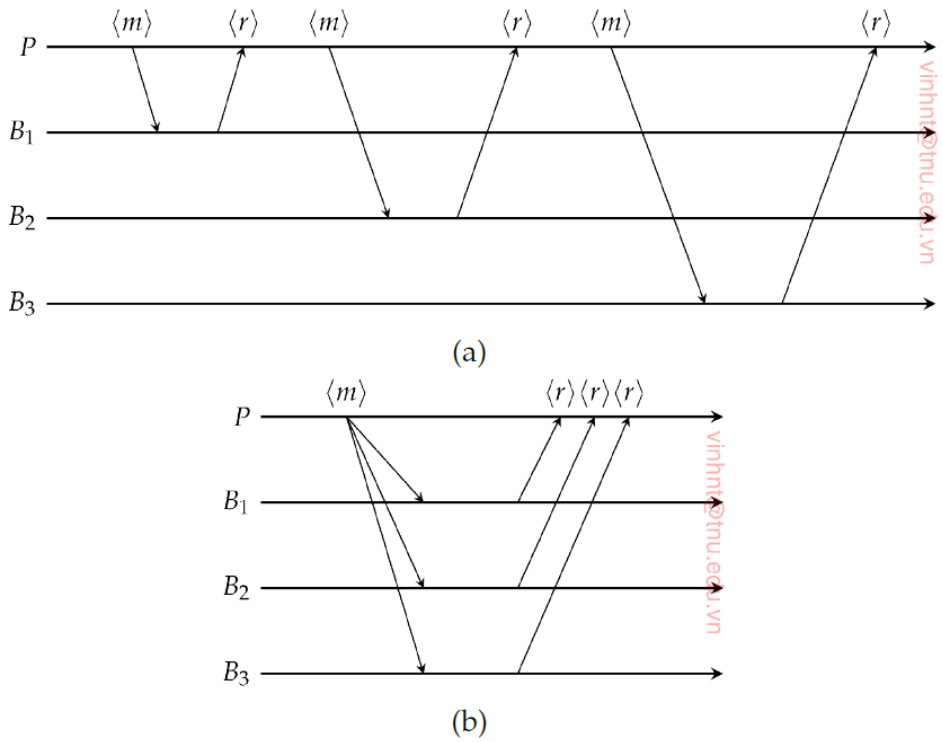
Figure 16: The distinction between receiving and delivering messages.

- **Receive:** The message-handling component gets the message from the network.
- **Deliver:** The message-handling component passes it to the core application functionality.

Reliability with Known Recipients

If we can ignore group membership consensus (i.e., the sender knows exactly who the intended recipients are), reliable communication is straightforward.

- The sender can deploy reliable transport-level protocols (like TCP) and send the message point-to-point to each recipient.
- To speed up communication, requests can be sent in parallel rather than sequentially waiting for responses.

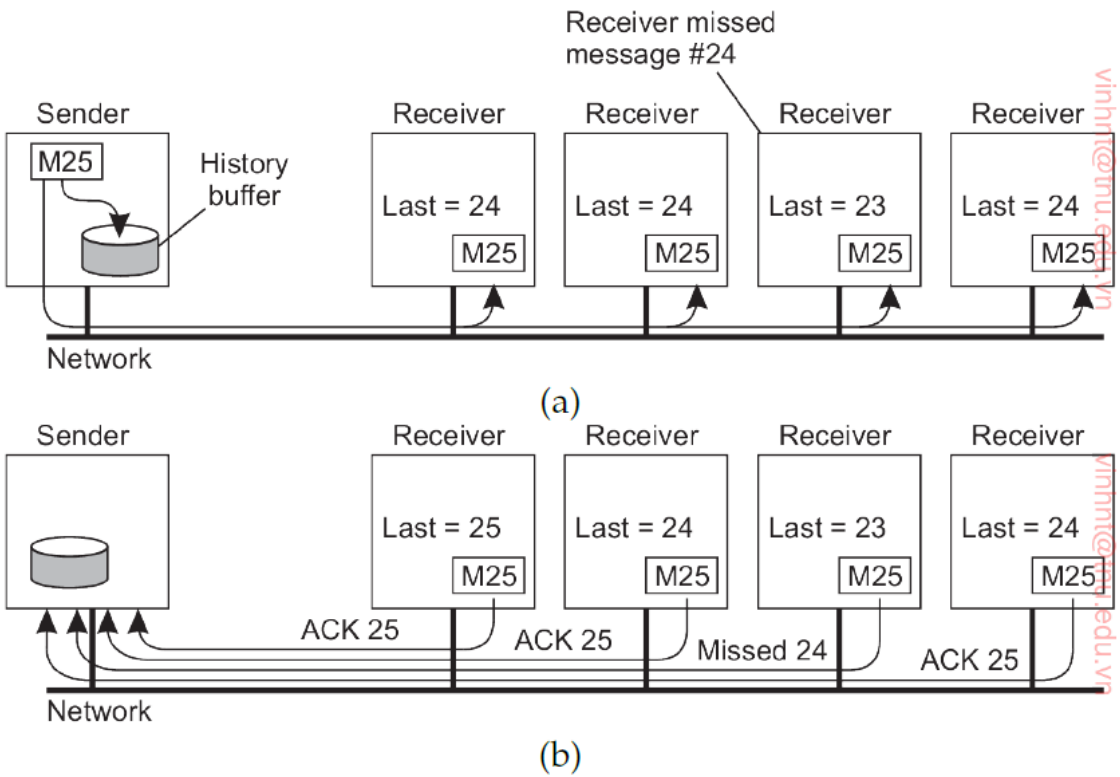


(a) Sequential requests with waiting (b) Parallel requests with responses

Basic Reliable Multicasting

If the underlying communication system offers unreliable multicasting, we must build reliability on top of it.

- The sender assigns a sequence number to each message and stores it in a history buffer.
- The receiver returns an acknowledgment (ACK) upon receipt.
- If a receiver detects a gap in sequence numbers, it sends a negative acknowledgment (NACK) requesting a retransmission.



A solution for reliable multicasting. (a) Message transmission. (b) Reporting feedback.

The Scalability Problem: Feedback Implosion

The basic reliable multicast scheme fails to scale to large groups.

Feedback Implosion

- If there are N receivers, the sender must process at least N acknowledgments.
- In large groups, the sender is quickly swamped by feedback messages.

Initial Mitigation

- Receivers only return NACKs (informing the sender only when a message is missed).
- *Trade-off:* The sender never knows for sure if a message was successfully delivered everywhere, theoretically forcing it to keep messages in the history buffer forever.

Nonhierarchical Feedback Control (Feedback Suppression)

The Problem (NACK Implosion): In reliable multicasting, if a sender drops a packet, thousands of receivers might simultaneously send a Negative Acknowledgement (NACK) back, overwhelming the sender's network and CPU.

The Solution (Feedback Suppression): To reduce these implosions, protocols like Scalable Reliable Multicasting (SRM) use a decentralized approach:

- Receivers multicast their NACKs to the **entire group**, not just the sender.
- Before sending a NACK, a receiver waits a random delay timer.
- If a receiver hears someone else's NACK for message m while its timer is active, it **suppresses** its own request, expecting the sender to retransmit shortly.
- *Drawbacks:* Accurate timer scheduling is difficult across wide-area networks (WANs), and multicasting NACKs interrupts "healthy" receivers with useless control traffic.

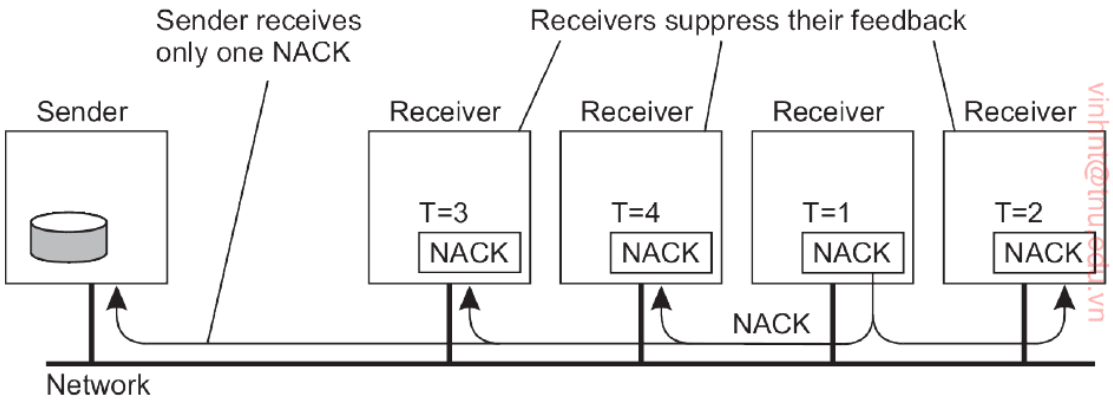
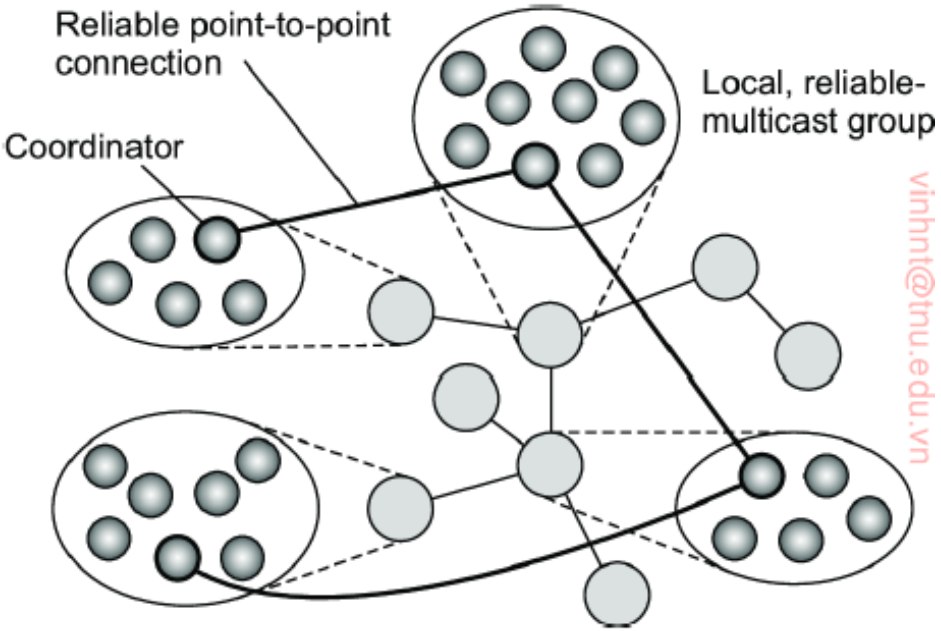


Figure 17: Several receivers schedule a NACK, but the first one transmitted suppresses the others.

Hierarchical Feedback Control

For exceptionally large groups, feedback suppression must be combined with a hierarchical approach.

- The group is partitioned into subgroups, organized into a tree.
- Each subgroup elects a **local coordinator** to represent it in the tree.
- Coordinators aggregate ACKs/NACKs and handle retransmissions locally, shielding the original sender.



vinhnt@tnu.edu.vn

Figure 18: The essence of hierarchical reliable multicasting. Each local coordinator forwards the message to its neighboring coordinators in the tree and later handles retransmission requests.

Atomic Multicast

In the presence of process failures, we need a strict guarantee: **Atomic Multicast**.

The Guarantee

- A message is delivered to either *all* nonfaulty group members, or to *none at all*.

Why is this crucial?

- Consider an actively replicated database. A client asks Replica P to perform an update.
- Replica P starts multicasting the update to the group, but crashes halfway through.
- If some replicas apply the update and others do not, the database state becomes inconsistent. Atomic multicast prevents this.

Virtual Synchrony: Aligning Messages with Membership

What is a View? A "View" (G) is simply the agreed-upon roster of currently active group members (e.g., $G_1 = \{A, B, C\}$).

Atomic multicast uses **Virtual Synchrony** to ensure message delivery perfectly aligns with changes to this roster:

- **The Synchronization Barrier:** A membership change (someone joins, leaves, or crashes) acts as a strict "pause" barrier.
- **View-Centric Delivery:** Every message belongs exclusively to the exact View in which it was sent. A message cannot cross the barrier into a new View.
- **The Flush Protocol:** Before the system updates the roster to a new view (G_{i+1}), all surviving members ping each other to share and deliver any pending, unstable messages from the old view (G_i).

The "All-or-Nothing" Roster Rule

A message sent during View G must be delivered to **all** surviving members of G before the roster updates. If the sender crashes before anyone else receives it, the message is permanently **discarded by all**.

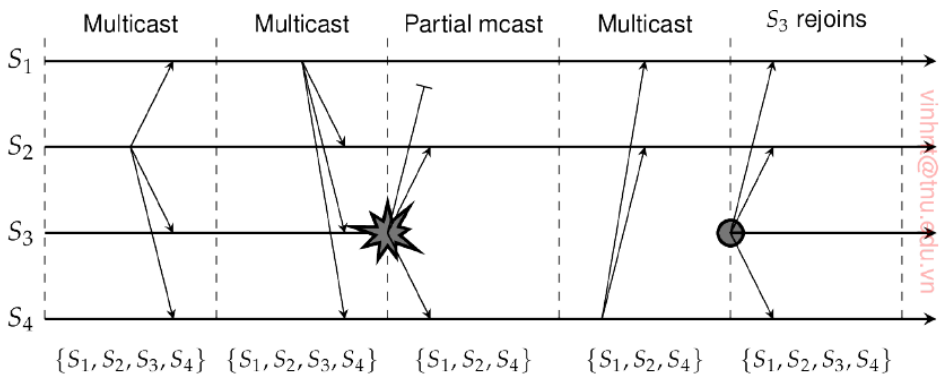


Figure 19: Virtual synchrony: View changes act as barriers to message delivery.

Message Ordering: Unordered Multicasts

Virtual synchrony guarantees delivery, but does not dictate the order in which messages are delivered.

Unordered Multicasts

- Reliable, virtually synchronous, but no constraints on the delivery sequence.
- Processes may deliver incoming messages in completely different orders.

Event order	Process P_1	Process P_2	Process P_3
1	sends m_1	receives m_1	receives m_2
2	sends m_2	receives m_2	receives m_1

Figure 20: Three communicating processes in the same group. The ordering of events per process is shown along the vertical axis.

FIFO-Ordered Multicasts

- Forces the communication layer to deliver incoming messages from the *same* sender in the exact order they were originally sent.
- However, messages sent by *different* processes can still be interleaved differently at each receiver.

Event order	Process P_1	Process P_2	Process P_3	Process P_4
1	sends m_1	receives m_1	receives m_3	sends m_3
2	sends m_2	receives m_3	receives m_1	sends m_4
3		receives m_2	receives m_2	
4		receives m_4	receives m_4	

Figure 21: Four processes in the same group with two different senders, and a possible delivery order of messages under FIFO-ordered multicasting.

Message Ordering: Causal and Total Ordering

Causally Ordered Multicasts

- Preserves potential causality. If message m_1 causally precedes m_2, all receivers must deliver m_1 before m_2, regardless of the sender.

Totally Ordered Delivery

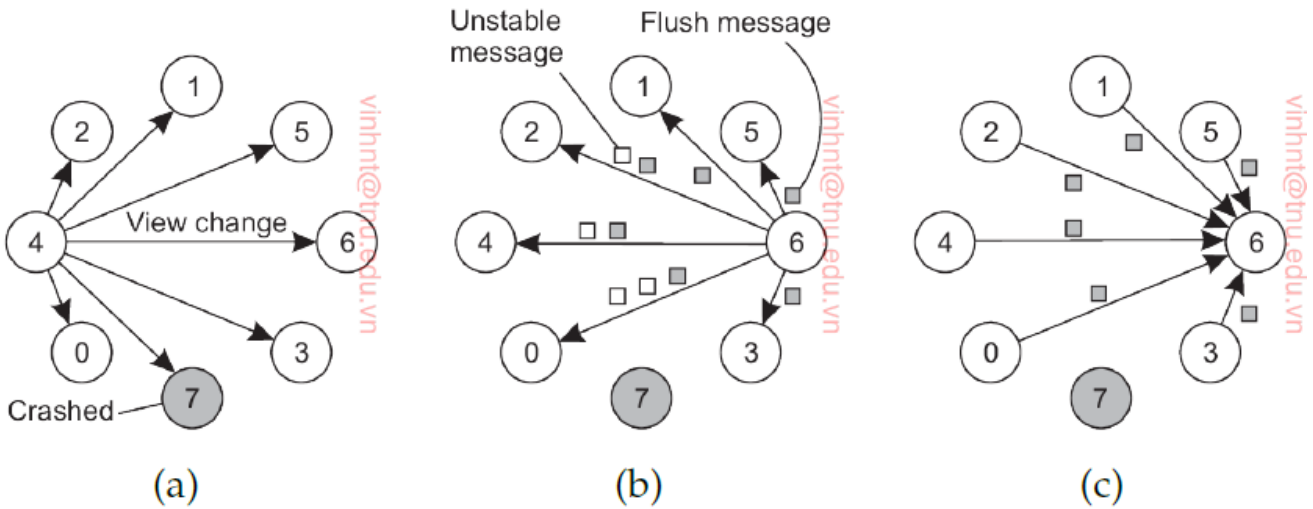
- Regardless of FIFO or Causal rules, total ordering demands that when messages are delivered, they are delivered in the *exact same sequence* to every group member.

Multicast	Basic message ordering	TO delivery?
Reliable multicast	None	No
FIFO multicast	FIFO-ordered delivery	No
Causal multicast	Causal-ordered delivery	No
Atomic multicast	None	Yes
FIFO atomic multicast	FIFO-ordered delivery	Yes
Causal atomic multicast	Causal-ordered delivery	Yes

Figure 22: Six different versions of virtually synchronous reliable multicasting and considering totally ordered delivery.

Implementing Virtual Synchrony

How do we ensure all nonfaulty processes receive a message before a view change if the sender crashes mid-multicast?



(a) Process 4 notices that process 7 has crashed and sends a view change. (b) Process 6 sends out all its unstable messages, followed by a flush message. (c) Process 6 installs the new view when it has received a flush message from everyone else.

The Flush Protocol

- When a process receives a view-change announcement, it forwards any *unstable* messages it holds to the rest of the group.
- Once finished, it sends a `flush` message.
- The new view is installed only when `flush` messages are received from all nonfaulty members.

Table of Contents

1. Introduction to fault tolerance
2. Process resilience
3. Reliable client-server communication
4. Reliable group communication
5. Distributed commit
6. Recovery
7. Summary

The Distributed Commit Problem

Distributed commit ensures that an operation is performed by *each* member of a process group, or by *none at all*.

One-Phase Commit Protocol

- A coordinator simply tells all participants whether or not to perform the operation.
- *Drawback:* If a participant cannot actually perform the operation (e.g., due to local constraints), there is no mechanism to tell the coordinator.

Two-Phase Commit Protocol (2PC)

- The most common sophisticated scheme to address the drawbacks of a one-phase commit.
- It relies on a voting phase followed by a decision phase.

Two-Phase Commit Protocol (2PC) Phases

Assuming no failures occur, 2PC consists of two distinct phases: **Phase 1: The Voting Phase**

- **Step 1:** The coordinator sends a `vote-request` message to all participants.
- **Step 2:** Participants return either a `vote-commit` (prepared to commit) or `vote-abort` message to the coordinator.

Phase 2: The Decision Phase

- **Step 3:** The coordinator collects votes. If all vote to commit, it multicasts `global-commit`. If even one votes to abort, it multicasts `global-abort`.
- **Step 4:** Participants waiting in the ready state receive the global message and locally commit or abort the transaction accordingly.

2PC Finite State Machines

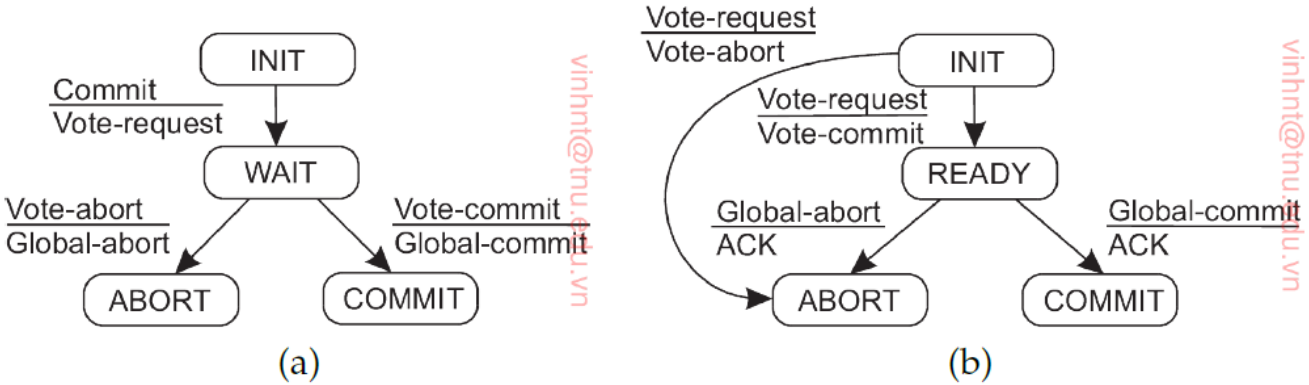


Figure 23: (a) The finite state machine for the coordinator in 2PC. (b) The finite state machine for a participant.

State Machine Observations:

- The protocol requires states where processes *block* waiting for incoming messages (e.g., Coordinator in WAIT, Participant in INIT or READY).
- Timeouts are essential to detect when a communicating party has crashed during these blocked states.

Handling Failures in 2PC

When processes crash, surviving processes must use timeouts to recover safely:

Participant in INIT state

- Waiting for a `vote-request`.
- *Timeout action:* Locally abort the transaction (the coordinator presumably crashed).

Coordinator in WAIT state

- Waiting for votes from all participants.
- *Timeout action:* Multicast `global-abort` (a participant presumably crashed).

The Blocking Problem: Participant in READY State

If a participant times out in the `READY` state, it cannot unilaterally decide to abort or commit. It must ask another participant (Q) what the coordinator sent.

State of Q	**Action by** P
COMMIT	Make transition to COMMIT
ABORT	Make transition to ABORT
INIT	Make transition to ABORT
READY	Contact another participant

Figure 24: Actions taken by a participant P when residing in state READY and having contacted another participant Q.

The Fatal Flaw of 2PC: If *all* operational participants are in the `READY` state, no decision can be made. The system blocks completely until the coordinator recovers.

2PC Coordinator Code Outline

Coordinator Responsibilities:

- Multicasts requests and manages the `yetToReceive` list.
- Enforces timeouts: If the timeout expires before all votes arrive, it forces a `GLOBAL_ABORT`.
- Logs its decisions to persistent storage before sending them, allowing it to retransmit upon recovery.

```
class Coordinator:

  def run(self):
    yetToReceive = list(participants)
    self.log.info('WAIT')
    self.chan.sendTo(participants, VOTE_REQUEST)
    while len(yetToReceive) > 0:
      msg = self.chan.recvFrom(participants, TIMEOUT)
      if (not msg) or (msg[1] == VOTE_ABORT):
        self.log.info('ABORT')
        self.chan.sendTo(participants, GLOBAL_ABORT)
        return
      else: # msg[1] == VOTE_COMMIT
        yetToReceive.remove(msg[0])
    self.log.info('COMMIT')
    self.chan.sendTo(participants, GLOBAL_COMMIT)
```

Figure 25: The steps taken by the coordinator in a 2PC protocol.

2PC Participant Code Outline

Participant Responsibilities:

- Evaluates local constraints (`do_work()`) before voting.
- Uses an infinite loop to assist other participants if the coordinator crashes, sharing the final decision if it is known.

```
class Participant:
  def run(self):
    self.log.info('INIT')
    msg = self.chan.recvFrom(coordinator, TIMEOUT)
    if (not msg):  # Crashed coordinator - give up entirely
      decision = LOCAL_ABORT
    else: # Coordinator will have sent VOTE_REQUEST
      decision = self.do_work()
      if decision == LOCAL_ABORT:
        self.chan.sendTo(coordinator, VOTE_ABORT)
      else: # Ready to commit, enter READY state
        self.log.info('READY')
        self.chan.sendTo(coordinator, VOTE_COMMIT)
        msg = self.chan.recvFrom(coordinator, TIMEOUT)
        if (not msg): # Crashed coordinator - check the others
          self.chan.sendTo(all_participants, NEED_DECISION)
          while True:
            msg = self.chan.recvFromAny()
            if msg[1] in [GLOBAL_COMMIT, GLOBAL_ABORT, LOCAL_ABORT]:
              decision = msg[1]
              break
        else: # Coordinator came to a decision
          decision = msg[1]
    if decision == GLOBAL_COMMIT:
      self.log.info('COMMIT')
    else: # decision in [GLOBAL_ABORT, LOCAL_ABORT]:
      self.log.info('ABORT')
    while True: # Help any other participant when coordinator crashed
      msg = self.chan.recvFrom(all_participants)
      if msg[1] == NEED_DECISION:
        self.chan.sendTo([msg[0]], decision)
```

The steps taken by a participant process in 2PC.

Three-Phase Commit Protocol (3PC)

To avoid the blocking problem inherent in 2PC, 3PC introduces an extra phase.

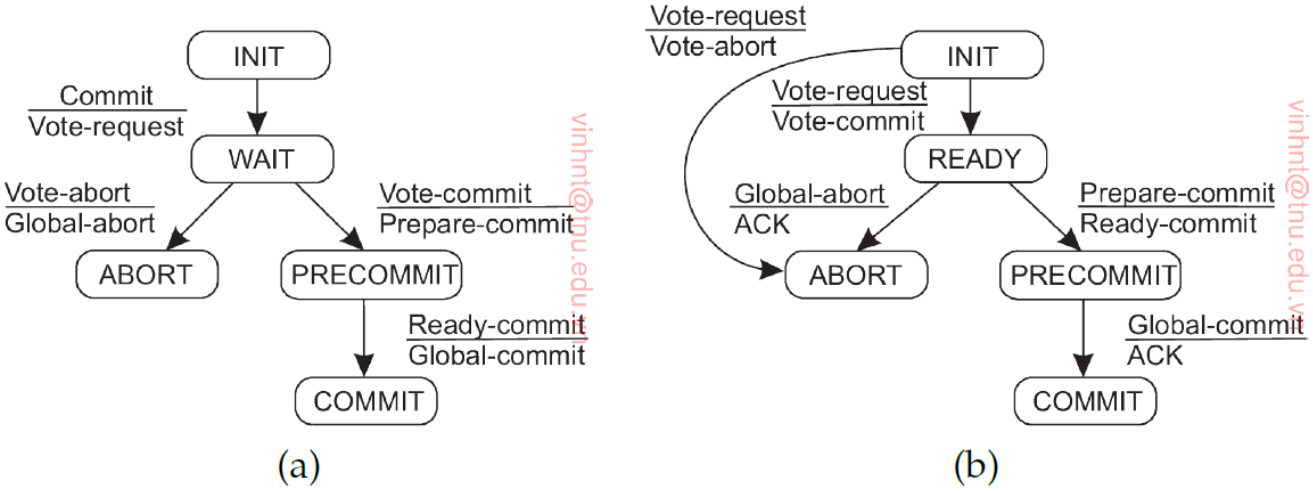


Figure 26: (a) The finite state machine for the coordinator in 3PC. (b) The finite state machine for a participant.

How 3PC Avoids Blocking:

- It adds a `PRECOMMIT` state.
- *Key Rule:* No participant can be in the `INIT` state while another is in `PRECOMMIT`.
- If the coordinator crashes and all surviving processes are in `READY`, they can safely abort, knowing no crashed process could have committed.

Table of Contents

1. Introduction to fault tolerance
2. Process resilience
3. Reliable client-server communication
4. Reliable group communication
5. Distributed commit
6. Recovery
7. Summary

Error Recovery Strategies

Once a failure occurs, the system must recover to an error-free state. **Backward Recovery**

- Brings the system back to a previously saved, correct state (a checkpoint).
- *Advantage:* Generally applicable and independent of the specific application.
- *Disadvantages:* Costly performance overhead, and some external actions (e.g., dispensing cash) cannot simply be rolled back.

Forward Recovery

- Attempts to move the system forward into a new correct state (e.g., using erasure correction to rebuild a lost packet instead of requesting retransmission).
- *Requirement:* The specific errors that might occur must be known in advance to design the correction mechanism.

Checkpointing and the Recovery Line

Backward recovery requires the system to periodically save a **distributed snapshot** (a consistent global state).

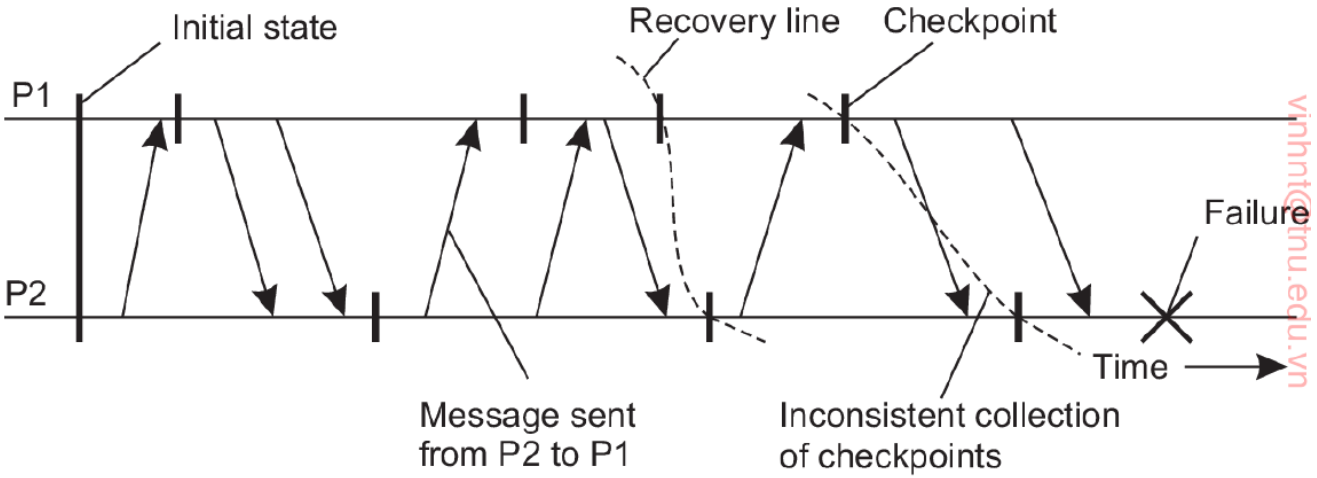


Figure 27: A recovery line.

The Recovery Line

- The most recent consistent collection of checkpoints across all processes.
- Consistency means if a process records receiving a message, another process must have recorded sending it.

Coordinated Checkpointing

In coordinated checkpointing, all processes synchronize to jointly write their state to local storage. **How it Works**

- Uses a two-phase blocking protocol.
- A coordinator multicasts a `checkpoint-request`.
- Processes take a local checkpoint, queue any new outgoing messages, and send an acknowledgment.
- Once all acknowledge, a `checkpoint-done` message allows processes to resume.

Incremental Snapshots

- To optimize, the coordinator only sends requests to processes it has communicated with since the last checkpoint, building the dependency graph dynamically.

Independent Checkpointing & The Domino Effect

If processes record their states independently without coordination, finding a consistent recovery line becomes highly complex.

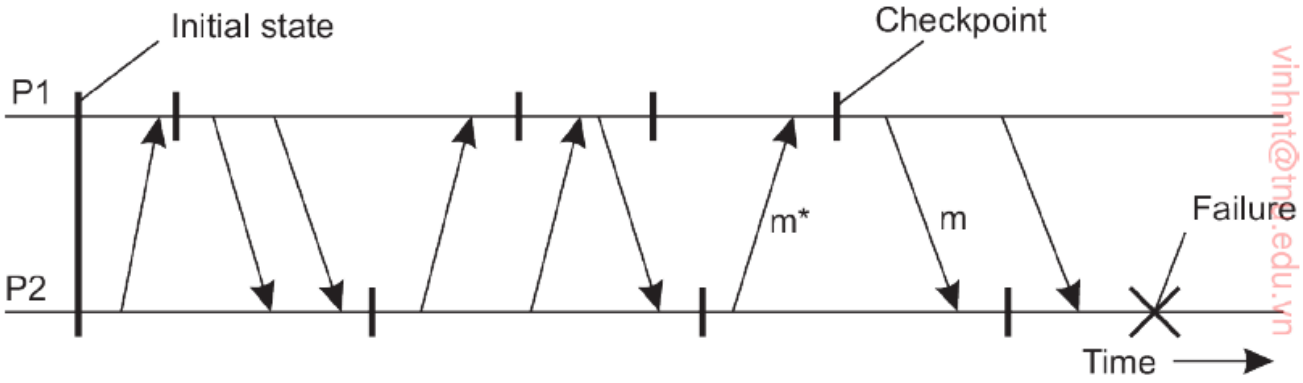


Figure 28: The domino effect.

vinhnt@tnu.edu.vn

The Domino Effect

- A cascaded rollback. If P_2's checkpoint shows it received a message that P_1 hasn't recorded sending yet, P_2 must roll back further.
- This can force P_1 to roll back, potentially unraveling all the way to the initial system state.
- *Conclusion:* Because of this overhead, coordinated checkpointing is generally preferred in practice.

Message Logging

To reduce the performance penalty of frequent checkpoints, systems combine occasional checkpoints with **message logging**. **Piecewise Deterministic Execution Model**

- Processes execute deterministically between nondeterministic events (like receiving a message).
- If we record all nondeterministic events (the messages), we can completely and deterministically replay the execution of a process after restoring an older checkpoint.

Advantage

- It allows restoring a state that lies *beyond* the most recent checkpoint, saving the cost of taking frequent snapshots.

Message Logging and Orphan Processes

The primary challenge in message logging is preventing the creation of orphan processes during recovery.

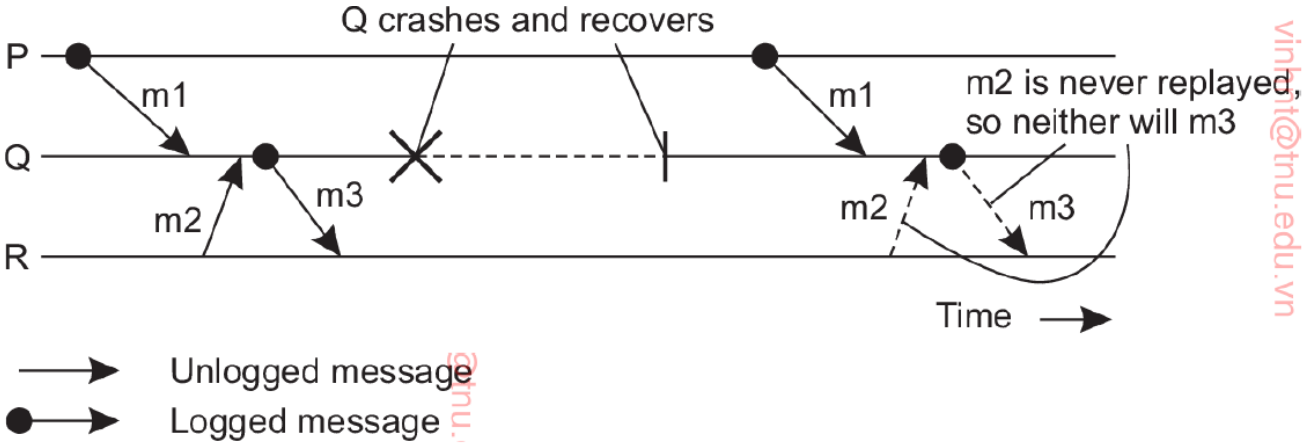


Figure 29: Incorrect replay of messages after recovery, leading to an orphan process R.

What is an Orphan Process?

- A process that survives a crash, but whose state is now inconsistent with the recovered process.
- In the figure, if Q does not log m_2, it will not replay sending m_3. Yet, process R still holds m_3, creating an impossible, inconsistent system state.

Pessimistic vs. Optimistic Logging

To prevent orphan processes, message logging schemes follow one of two paradigms: **Pessimistic Logging (Industry Preferred)**

- Guarantees that a nonstable message is delivered to at most one process.
- A process is *not allowed* to send any new messages until it ensures the previous incoming message is written to reliable storage.
- Orphans are structurally impossible, making implementation simple.

Optimistic Logging

- Does the work *after* a crash occurs.
- Messages are logged asynchronously. If a crash creates an orphan, the orphan process is forced to roll back to a prior, consistent state.
- Much more complex to implement due to deep dependency tracking.

Recovery-Oriented Computing

A paradigm shift: Instead of striving for zero failures, optimize the system to recover instantly. **Micro-Rebooting**

- Localize the fault to a specific component.
- Reboot only that component by deleting its threads and restarting the associated requests, leaving the rest of the server intact.
- Requires highly decoupled software architectures.

Environment Mutation

- When restarting software, change its execution environment slightly (e.g., zero the memory, allocate more buffer space, change message delivery order).
- Because software failures are highly deterministic, slightly altering the environment often bypasses the bug without actually repairing the code.

Table of Contents

1. Introduction to fault tolerance
2. Process resilience
3. Reliable client-server communication
4. Reliable group communication
5. Distributed commit
6. Recovery
7. Summary

Core Concepts and Failure Models

Fault Tolerance Defined

- The characteristic by which a system can mask the occurrence of failures and automatically recover, ensuring continuous operation.

Types of Failures

- *Crash:* A process simply halts.
- *Omission:* A process does not respond to requests.
- *Timing:* A process responds too soon or too late.
- *Response:* A process responds incorrectly.
- *Arbitrary (Byzantine):* A process exhibits any kind of failure, including malicious behavior.

The Role of Redundancy

- Achieved primarily through **process groups**, where identical processes cooperate so that if one fails, others seamlessly take over the service.

Consensus and Reliable Communication

Reaching Consensus

- Process groups must reach consensus on operations. Algorithms like **Paxos** are highly robust solutions for this.
- *Thresholds:* $2k + 1$ servers are needed to tolerate k crash failures; $3k + 1$ servers are needed to tolerate k arbitrary failures.

Reliable Group Communication

- *Scalability:* Reliable multicasting is feasible for small groups but struggles in large groups due to feedback (ACK/NACK) implosions.
- *Atomicity:* Requires that all group members have the exact same view of delivered messages.
- *Virtual Synchrony:* An execution model that enforces atomicity by creating strict boundaries around group membership changes where messages cannot cross.

Distributed Commit and System Recovery

Distributed Commit Protocols

- Required when processes must agree on operations, such as group membership changes.
- **Two-Phase Commit (2PC):** Widely applied; checks for agreement then multicasts the outcome. However, it can block if the coordinator crashes.
- **Three-Phase Commit (3PC):** Adds an extra phase to handle coordinator crashes without forcing the entire system to block.

System Recovery

- **Checkpointing:** Saving the distributed state to local storage regularly. It is completely distributed but computationally expensive.
- **Message Logging:** To improve performance, systems combine occasional checkpoints with message logs, allowing them to precisely *replay* the execution history after a crash.